

Interfaces II

Listas, entrada de dados e a tela que responde

Prof. Cristiano Rodrigues

O número cresce no terminal e não muda na tela. Por quê?



Alguém arrisca?

Para acompanhar hoje

Você precisa saber

- Compor uma tela com Column, Row e Expanded
- Que o build descreve a tela, e que ele roda quando alguém pede
- Usar as cores do tema em vez de escrevê-las à mão

Você precisa ter feito

- A lista de hábitos e a tela de detalhe rodando no seu projeto
- A versão de treino do Check, publicada no Canvas

PARTE 01

Listas que crescem

Do jeito que funciona com dez itens ao jeito que funciona com mil.

Duas formas de montar uma lista

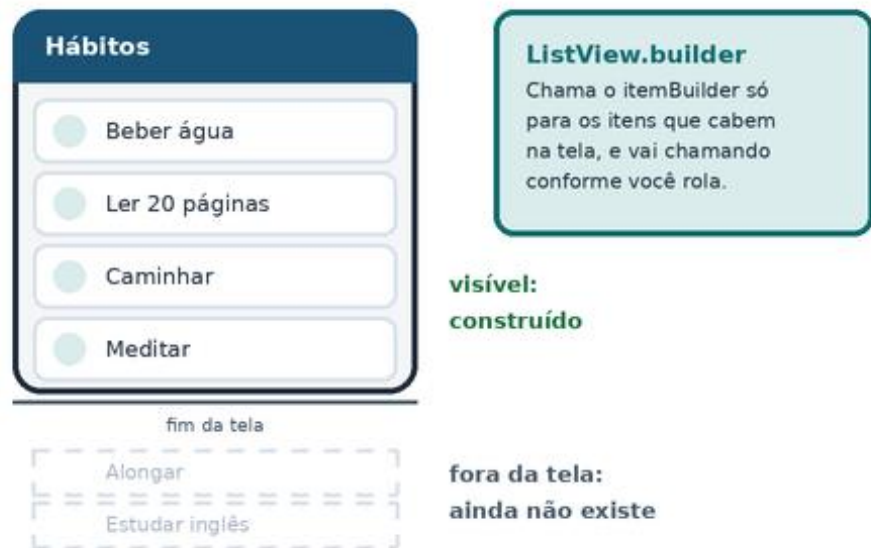
ListView(children: [...])

- Constrói todos os itens de uma vez.
- Simples e legível.
- Serve até algumas dezenas de itens.
- Com mil itens, o aplicativo trava.

ListView.builder

- Constrói só o que está visível.
- Exige itemCount e itemBuilder.
- Serve para qualquer tamanho.
- Deve ser o seu padrão.

O builder só constrói o que cabe na tela



Os itens fora da área visível ainda não foram construídos.

A lista com separador entre os itens

```
ListView.separated(  
  itemCount: habitos.length,           // quantos itens existem  
  separatorBuilder: (context, i) => const Divider(height: 1),  
  itemBuilder: (context, i) {          // constrói o item de índice i  
    final h = habitos[i];  
    return ListTile(  
      leading: Icon(h.icone),  
      title: Text(h.nome),  
      subtitle: Text(h.meta),  
    );  
  },  
)
```

lib/tela_habitos.dart · separated é o builder com uma linha divisória entre os itens

PARTE 02

A tela que responde

O que muda quando o aplicativo passa a ter memória do que aconteceu.

O contador que não conta

```
class Contador extends StatelessWidget {  
  const Contador({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    int copos = 0;                                // some a cada build  
    return FilledButton(  
      onPressed: () => copos++,                    // o número cresce...  
      child: Text('$copos copos'),                // ...e a tela não muda  
    );  
  }  
}
```

lib/contador.dart · compila, roda, e não funciona

Porque ninguém mandou desenhar de novo

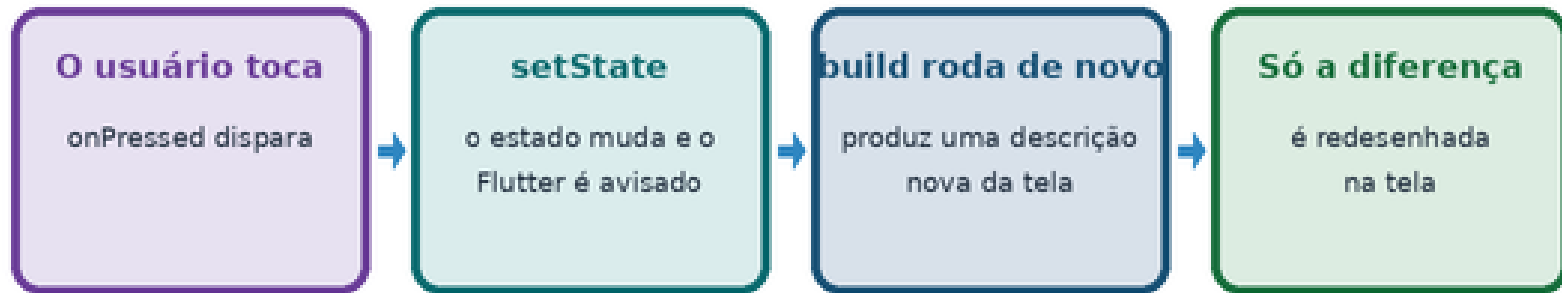
O build não roda sozinho. Ele rodou uma vez, produziu a descrição da tela, e acabou. Alterar a variável depois disso não avisa ninguém.

setState é o aviso. Ele diz ao Flutter que o estado desta tela mudou e que o build precisa rodar de novo.

A regra continua a mesma: você não altera a tela. Você altera o estado e pede uma nova descrição.

O que setState dispara

O que acontece quando você chama setState



Alterar a variável fora do setState para no primeiro passo: ninguém é avisado.

Do toque do usuário até a tela redesenhada.

O mesmo contador, agora com estado

```
class Contador extends StatefulWidget {  
  @override  
  State<Contador> createState() => _ContadorState();  
}  
  
class _ContadorState extends State<Contador> {  
  int _copos = 0;                // sobrevive ao build  
  @override  
  Widget build(BuildContext context) => FilledButton(  
    onPressed: () => setState(() => _copos++),  
    child: Text('$ _copos copos'),  
  );  
}
```

lib/contador.dart · duas classes: o widget é descartado, o State permanece

setState não atualiza a tela.

Ele avisa que o estado mudou e pede uma nova construção. A diferença parece sutil e não é: alterar a variável fora do setState não muda a tela, e chamar setState sem alterar nada faz o Flutter reconstruir e não encontrar diferença alguma.



Quando uma tela precisa virar Stateful

A pergunta

Existe algo na tela que muda sem o usuário sair dela?

O caminho

Comece Stateless. Converta quando a resposta virar sim.

O custo

Stateful sem necessidade não quebra nada, mas é peso que não paga.

No VS Code, a lâmpada converte a classe inteira: Convert to StatefulWidget.

PARTE 03

Entrada de dados

Capturar o que o usuário digita, e recusar o que não serve.

Capturar o que o usuário digita

```
final _controle = TextEditingController(); // campo do State

@override
void dispose() {
  _controle.dispose(); // libere sempre
  super.dispose();
}

TextField(controller: _controle)

// em qualquer ponto: _controle.text
```

lib/tela_habitos.dart · o controller guarda o texto e sobrevive à reconstrução

Validar antes de aceitar

```
final _chave = GlobalKey<FormState>(); // sem ela, validate falha
Form(
  key: _chave,
  child: TextFormField(
    validator: (v) {
      if (v == null || v.trim().isEmpty) return 'Informe o nome';
      if (v.trim().length < 3) return 'Use ao menos 3 letras';
      return null; // null significa válido
    },
  ),
)

if (_chave.currentState!.validate()) { /* pode salvar */ }
```

lib/tela_habitos.dart · validate percorre todos os campos do Form de uma vez

O validator devolve a mensagem, não o veredito

validator devolve uma mensagem

Nome do hábito

Use ao menos 3 letras

validate() devolve false

validator devolve null

Nome do hábito

validate() devolve true

Parece invertido: a mensagem é o erro, e null é o sinal de que está tudo certo.

À esquerda, campo inválido. À direita, campo válido.

Três regras de validação que valem para sempre

Onde

Valide na entrada. O erro aparece embaixo do campo, não numa caixa depois.

O quê

Use trim antes de verificar. Espaço em branco não é conteúdo.

Como

Diga o que fazer, não o que está errado. A mensagem é instrução.

A terceira volta como critério de usabilidade na Sprint do TI-IV.

A prática de agora, tudo numa tela só

Meus Hábitos

Novo hábito

Nome
Meditar

Meta diária
10 minutos

Adicionar

3 hábitos

- Beber água
8 copos por dia
- Ler
20 páginas por dia
- Caminhar
30 minutos por dia

- **Converta a tela da lista** em StatefulWidget e guarde a lista no State.
- **Acrescente o formulário** acima da lista, com dois campos validados.
- **Adicionar** insere o hábito na lista, dentro do setState.
- **O hábito some ao reiniciar.** É esperado: nada foi gravado.

Prática guiada, feita junto com o professor.

Os outros componentes de tela estão no Canvas

Entrada de dados

Caixa de seleção · Escolha única · Chave liga/desliga
Controle deslizante · Lista suspensa · Data e hora

Navegação estrutural

Menu lateral · Barra inferior · Abas
Botão de ação principal · Barra superior com ações

Catálogo de componentes: o código mínimo, para que serve e a armadilha de cada um.

PARTE 04

Check Individual 1

Nesta aula, você aprendeu a:

- Construir listas longas sem repetir código, com `ListView.builder`
- Explicar por que alterar uma variável não muda o que aparece na tela
- Guardar e atualizar estado dentro de uma tela com `setState`
- Capturar o que o usuário digita e liberar o controller ao sair
- Validar um formulário e mostrar a mensagem de erro no campo certo

Referências

Flutter. Work with long lists. Disponível em:
docs.flutter.dev/cookbook/lists/long-lists.

Flutter. Add interactivity to your Flutter app. Disponível em:
docs.flutter.dev/ui/interactivity.

Flutter. Build a form with validation. Disponível em:
docs.flutter.dev/cookbook/forms/validation.

Catálogo de componentes de interface. Deck de referência da disciplina, no Canvas.